

ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No.34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

Database Management Systems

AIRLINE RESERVATION SYSTEM

A Relational Database Design for Flight Booking & Management

Bachelor of Computer Application
Semester – III

R MONISHA BANGERA
2511021240053

Submitted To -

Faculty Name: Renita Joy R

Faculty Signature:

1. Objective

Data integrity means the data in the database is accurate, consistent and valid at all times. Instead of trusting every application to check its input, the Airline Reservation System enforces its rules inside the database using constraints. This report lists the constraints implemented in `airline_reservation_system.sql`, explains why each one is needed, and describes the tests in `data_integrity_tests.sql` that verify them.

2. Types of Constraints Used

Constraint	Purpose	Integrity type
PRIMARY KEY	Uniquely identifies each row; cannot be NULL or duplicated	Entity integrity
FOREIGN KEY	Ensures a value in a child table matches an existing row in the parent table	Referential integrity
NOT NULL	Forces a value to be supplied for mandatory columns	Completeness
UNIQUE	Prevents duplicate values in a column or combination of columns that must be distinct	Uniqueness
CHECK	Restricts values to a valid range or list	Domain integrity
DEFAULT	Supplies a standard value when none is given	Consistency

3. Primary Keys

Every table has a single-column integer primary key declared `AUTO_INCREMENT`, so the database generates unique identifiers automatically.

Table	Primary key
Passenger	Passenger_ID
Airport	Airport_ID
Aircraft	Aircraft_ID
Flight	Flight_ID
Seat	Seat_ID
Booking	Booking_ID
Ticket	Ticket_ID
Payment	Payment_ID
Baggage	Baggage_ID

4. Foreign Keys

Foreign keys implement the relationships of the ER diagram. By default they use RESTRICT, so a parent row cannot be deleted or have its key changed while child rows depend on it. This protects the history of bookings, payments and tickets.

Child table.column	Parent table.column	Constraint name	Relationship
Flight.Aircraft_ID	Aircraft.Aircraft_ID	fk_flight_aircraft	Aircraft 1 : M Flight
Flight.Departure_Airport_ID	Airport.Airport_ID	fk_flight_departure	Airport 1 : M Flight (departure)
Flight.Arrival_Airport_ID	Airport.Airport_ID	fk_flight_arrival	Airport 1 : M Flight (arrival)
Seat.Aircraft_ID	Aircraft.Aircraft_ID	fk_seat_aircraft	Aircraft 1 : M Seat
Booking.Passenger_ID	Passenger.Passenger_ID	fk_booking_passenger	Passenger 1 : M Booking
Booking.Flight_ID	Flight.Flight_ID	fk_booking_flight	Flight 1 : M Booking
Booking.Seat_ID	Seat.Seat_ID	fk_booking_seat	Seat 1 : M Booking
Ticket.Booking_ID	Booking.Booking_ID	fk_ticket_booking	Booking 1 : 1 Ticket

Child table.column	Parent	Constraint name	Relationship
Payment.Booking_ID	table.column	fk_payment_booking	Booking 1 : M Payment
Baggage.Booking_ID	Booking.Booking_I	fk_baggage_booking	Booking 1 : 1 Baggage

D

Booking.Booking_I

D

Because Booking holds foreign keys to both Passenger and Flight, it resolves the many-to-many relationship between them.

5. UNIQUE Constraints

Table	Column(s)	Why it must be unique
Passenger	Email	One person should not be registered twice with the same email
Airport	Airport_Code	An IATA code such as BLR identifies exactly one airport
Aircraft	Registration_No	Each physical aircraft has one registration
Flight	(Flight_Number, Departure_DateTime)	A flight number can fly only once at a given departure time
Seat	(Aircraft_ID, Seat_Number)	Seat 12A exists only once on an aircraft
Booking	(Flight_ID, Seat_ID)	A seat can be sold only once per flight, which prevents double booking
Ticket	Booking_ID	Makes Booking-Ticket truly 1:1
Ticket	Ticket_Number	Printed ticket numbers must not repeat
Paymen t	Transaction_ID	One transaction reference cannot be recorded twice
Baggage	Booking_ID	Makes Booking-Baggage truly 1:1

6. CHECK Constraints

Table	Constraint	Rule	Why
Passenger	chk_passenger_gender	Male, Female or Other (or NULL)	Keeps gender values consistent
Airport	chk_airport_terminals	Terminal_Count > 0	An airport has at least one terminal
Aircraft	chk_aircraft_seats	Total_Seats > 0	Capacity cannot be zero or negative
Aircraft	chk_aircraft_status	Active or Maintenance	Only valid statuses
Flight	chk_flight_airports	Departure airport differs from arrival airport	A flight cannot start and end at the same airport
Flight	chk_flight_times	Arrival after departure	Time cannot run backwards
Flight	chk_flight_status	Scheduled, Delayed or Cancelled	Only valid statuses
Seat	chk_seat_class	Economy, Business or First	Only valid classes
Seat	chk_seat_status	Available or Reserved	Only valid statuses
Booking	chk_booking_status	Confirmed, Cancelled or Pending	Only valid statuses
Booking	chk_booking_fare	Fare >= 0	No negative fares
Ticket	chk_ticket_status	Valid or Cancelled	Only valid statuses
Payment	chk_payment_method	Card, UPI or Net Banking	Only supported methods
Payment	chk_payment_status	Successful, Failed or Pending	Only valid statuses
Payment	chk_payment_amount	Amount > 0	A payment must be a positive sum

Table	Constraint	Rule	Why
Baggage	chk_baggage_values	Count, weight and fee all >= 0	No negative quantities

7. DEFAULT and NOT NULL Constraints

Table	Column	Default	Purpose
Airport	Terminal_Count	1	Every airport has at least one terminal
Aircraft	Aircraft_Status	'Active'	New aircraft are assumed in service
Flight	Flight_Status	'Scheduled'	New flights start as scheduled
Seat	Seat_Class	'Economy'	Most seats are economy
Seat	Seat_Status	'Available'	New seats are free
Booking	Booking_Status	'Pending'	A booking is pending until payment succeeds
Ticket	Ticket_Status	'Valid'	A newly issued ticket is valid
Payment	Payment_Status	'Pending'	Payment starts as pending
Baggage	Baggage_Count, Total_Weight, Excess_Baggage_Fee	0	No baggage unless recorded

`NOT NULL` is applied to all primary keys, all foreign keys and every business-critical column (names, email, phone, dates, fare, amount, flight times). This guarantees that no booking exists without a passenger, flight and seat.

8. How the Constraints Were Verified

The script `data_integrity_tests.sql` has three parts.

Part A: valid data. Sample passengers, airports, aircraft, flights, seats, bookings, a ticket, a payment and a baggage record are inserted successfully. A second group of inserts omits the defaulted columns, and a query then shows the DEFAULT values were applied.

Part B: invalid data. Thirty-nine statements deliberately break a rule. Each is expected to be rejected with an error:

Rule tested	Example statement	Expected error
Primary key	Insert Passenger_ID 1 again	1062 Duplicate entry
NOT NULL	Insert a passenger with NULL First_Name	1048 Column cannot be null
UNIQUE	Reuse an email, airport code or registration number	1062 Duplicate entry
UNIQUE (double booking)	Book seat 1 on flight 101 a second time	1062 Duplicate entry
UNIQUE (1:1)	Create a second ticket or baggage record for booking 5001	1062 Duplicate entry
CHECK	Gender 'X', 0 terminals, negative seats, arrival before departure, fare of -100, payment method 'Cash'	3819 Check constraint violated
FOREIGN KEY (child)	Flight with aircraft 99, booking for passenger 99, ticket for booking 9999	1452 Cannot add or update a child row
FOREIGN KEY (parent)	Delete passenger 1, booking 5001, flight 101 or seat 1 while child rows exist	1451 Cannot delete or update a parent row

The same seat on a different flight, and the same seat number on a different aircraft, are allowed. This confirms the constraints block only genuinely invalid data.

Part C: positive verification. After the failed statements, queries confirm that nothing was changed:

- Row counts per table match the valid data only (2, 3, 3, 4, 5, 4, 1, 1, 2).
- A nine-table join returns the complete record of booking 5001 (passenger, route, aircraft, seat, ticket, payment, baggage), showing the relationships work.

- Orphan-check queries (child rows with no parent) return zero rows.
- The double-booking query returns zero rows.
- INFORMATION_SCHEMA queries list every constraint and default stored in the database.

9. Known Limitations

- A cancelled booking still occupies its (Flight_ID, Seat_ID) pair, so the seat cannot be rebooked unless the old booking is removed or reassigned.
- SQL cannot prevent a booking from using a seat belonging to a different aircraft than the flight's, without adding the aircraft to Booking or a per-flight seat table.
- Derived values (Total_Seats, Payment.Amount, Excess_Baggage_Fee) are not checked against the data they come from; triggers or application logic would be needed.

10. Conclusion

Primary keys give every record an identity, foreign keys keep the nine tables consistent with one another, UNIQUE constraints stop duplicates and double bookings, CHECK constraints keep values inside their valid domains, DEFAULT values standardize new records, and NOT NULL ensures mandatory data is always present. The test script shows that valid data is accepted and invalid data is rejected by the database itself, so the integrity of the Airline Reservation System does not depend on any single application.

SQL Code

```sql
-- ========================================================================
-- PART A : VALID DATA (every statement should succeed)
-- ========================================================================
INSERT INTO Passenger (Passenger_ID, First_Name, Last_Name, Email, Phone,
Date_of_Birth, Gender) VALUES
 (1, 'Monisha', 'Bangera', 'monisha@example.com', '9876543210', '2005-03-14',
'Female'),
 (2, 'Rahul',  'Sharma',  'rahul@example.com',  '9123456780', '1999-07-21',
'Male');

INSERT INTO Airport (Airport_ID, Airport_Code, Airport_Name, City, Country,
Terminal_Count) VALUES
 (1, 'BLR', 'Kempegowda International Airport',  'Bengaluru',
'India', 2),
 (2, 'DEL', 'Indira Gandhi International Airport',  'Delhi',
'India', 3),
 (3, 'BOM', 'Chhatrapati Shivaji Maharaj International Airport', 'Mumbai',
'India', 2);

INSERT INTO Aircraft (Aircraft_ID, Aircraft_Model, Registration_No,
Total_Seats, Aircraft_Status) VALUES
 (1, 'Airbus A320',  'VT-ABC', 180, 'Active'),
 (2, 'Boeing 737-800',  'VT-XYZ', 189, 'Maintenance');

INSERT INTO Flight (Flight_ID, Flight_Number, Aircraft_ID,
Departure_Airport_ID, Arrival_Airport_ID,
                    Departure_DateTime, Arrival_DateTime, Flight_Status)
VALUES
 (101, 'AI502', 1, 1, 2, '2026-11-01 06:00:00', '2026-11-01 08:45:00', 'Scheduled'),
 (102, 'AI502', 1, 1, 2, '2026-11-02 06:00:00', '2026-11-02 08:45:00', 'Scheduled'),
 (103, '6E204', 1, 1, 3, '2026-11-01 09:30:00', '2026-11-01 11:10:00', 'Delayed');

INSERT INTO Seat (Seat_ID, Aircraft_ID, Seat_Number, Seat_Class, Seat_Status) VALUES
 (1, 1, '12A', 'Economy',
 (2, 1, '12B', 'Economy',


                                'Available'),
                                'Available'),
 (3, 1, '1A',  'Business', 'Available'),
 (4, 2, '12A', 'Economy',  'Available');  -- same seat number, different
aircraft: allowed

INSERT INTO Booking (Booking_ID, Passenger_ID, Flight_ID, Seat_ID, Booking_Date,
Booking_Status, Fare) VALUES
 (5001, 1, 101, 1, '2026-10-23', 'Confirmed', 5500.00),
 (5002, 2, 101, 2, '2026-10-23', 'Confirmed', 5500.00),
 (5003, 1, 102, 1, '2026-10-23', 'Confirmed', 5200.00);  -- seat 1 on a
DIFFERENT flight: allowed

```

```sql
INSERT INTO Ticket (Ticket_ID, Booking_ID, Ticket_Number, Issue_Date,
Ticket_Status) VALUES
 (1, 5001, 'TKT-5001-AI502', '2026-10-23', 'Valid');

INSERT INTO Payment (Payment_ID, Booking_ID, Payment_Date, Payment_Method,
Amount, Payment_Status, Transaction_ID) VALUES
 (1, 5001, '2026-10-23', 'UPI', 5500.00, 'Successful', 'UPI-TXN-0001');

INSERT INTO Baggage (Baggage_ID, Booking_ID, Baggage_Count, Total_Weight,
Excess_Baggage_Fee) VALUES
 (1, 5001, 1, 15.50, 0.00);

-- ---- DEFAULT constraint demonstration: omit the defaulted columns ----
INSERT INTO Aircraft (Aircraft_Model, Registration_No, Total_Seats)
VALUES ('ATR 72', 'VT-DEF', 78);                                    --
Aircraft_Status -> 'Active'

INSERT INTO Seat (Aircraft_ID, Seat_Number)
VALUES (1, '14C');                                                  --
Seat_Class -> 'Economy', Seat_Status -> 'Available'

INSERT INTO Flight (Flight_Number, Aircraft_ID, Departure_Airport_ID,
Arrival_Airport_ID,
                                Departure_DateTime, Arrival_DateTime)
VALUES ('AI999', 1, 2, 1, '2026-11-05 10:00:00', '2026-11-05 12:40:00');                                    --
Flight_Status -> 'Scheduled'

INSERT INTO Booking (Passenger_ID, Flight_ID, Seat_ID, Booking_Date, Fare)
VALUES (2, 102, 2, '2026-10-23', 5200.00);                          --
Booking_Status -> 'Pending'

INSERT INTO Baggage (Booking_ID) VALUES (5002);                     -- count,
weight, fee -> 0

SELECT 'Aircraft default' AS Test, Aircraft_Model AS Item, Aircraft_Status AS
Default_Value
FROM Aircraft WHERE Registration_No = 'VT-DEF'
UNION ALL
SELECT 'Seat default', Seat_Number, CONCAT(Seat_Class, ' / ', Seat_Status)
FROM Seat WHERE Seat_Number = '14C'
UNION ALL
SELECT 'Flight default', Flight_Number, Flight_Status
FROM Flight WHERE Flight_Number = 'AI999'
UNION ALL
SELECT 'Booking default', CONCAT('Booking ', Booking_ID), Booking_Status
FROM Booking WHERE Booking_ID > 5003
UNION ALL
SELECT 'Baggage default', CONCAT('Booking ', Booking_ID),
          CONCAT(Baggage_Count, ' bags / ', Total_Weight, ' kg / fee ',
Excess_Baggage_Fee)
FROM Baggage WHERE Booking_ID = 5002;

-- ================================================================
-- PART B : INVALID DATA (every statement is EXPECTED TO FAIL)
```

```
-- ========================================================================

-- ---------- PRIMARY KEY ----------
-- B1 Expected: 1062 Duplicate entry '1' for key 'Passenger.PRIMARY'
INSERT INTO Passenger (Passenger_ID, First_Name, Last_Name, Email, Phone, Date_of_Birth)
VALUES (1, 'Test', 'User', 'test1@example.com', '9000000001', '2000-01-01');
-- ---------- NOT NULL ----------
-- B2 Expected: 1048 Column 'First_Name' cannot be null
INSERT INTO Passenger (First_Name, Last_Name, Email, Phone, Date_of_Birth) VALUES (NULL,
'User', 'test2@example.com', '9000000002', '2000-01-01');

-- ---------- UNIQUE ----------
-- B3 Expected: 1062 Duplicate entry 'monisha@example.com' for key 'Passenger.Email'
INSERT INTO Passenger (First_Name, Last_Name, Email, Phone, Date_of_Birth) VALUES ('Another',
'Person', 'monisha@example.com', '9000000003', '2000-01- 01');

-- B4 Expected: 1062 Duplicate entry 'BLR' for key 'Airport.Airport_Code' INSERT INTO Airport
(Airport_Code, Airport_Name, City, Country, Terminal_Count)
VALUES ('BLR', 'Fake Airport', 'Nowhere', 'India', 1);

-- B5 Expected: 1062 Duplicate entry 'VT-ABC' for key 'Aircraft.Registration_No'
INSERT INTO Aircraft (Aircraft_Model, Registration_No, Total_Seats)
VALUES ('Airbus A321', 'VT-ABC', 200);

-- B6 Expected: 1062 duplicate on (Flight_Number, Departure_DateTime) INSERT INTO Flight
(Flight_Number, Aircraft_ID, Departure_Airport_ID, Arrival_Airport_ID,

                                Departure_DateTime, Arrival_DateTime)
VALUES ('AI502', 1, 1, 2, '2026-11-01 06:00:00', '2026-11-01 09:00:00');

-- B7 Expected: 1062 duplicate on (Aircraft_ID, Seat_Number): seat 12A
already exists on aircraft 1
INSERT INTO Seat (Aircraft_ID, Seat_Number) VALUES (1, '12A');

-- B8 DOUBLE BOOKING. Expected: 1062 duplicate on (Flight_ID, Seat_ID): seat
1 on flight 101 is taken
INSERT INTO Booking (Passenger_ID, Flight_ID, Seat_ID, Booking_Date,
Booking_Status, Fare)
VALUES (2, 101, 1, '2026-10-23', 'Confirmed', 5500.00);

-- B9 1:1 rule. Expected: 1062 duplicate on Ticket.Booking_ID (booking 5001
already has a ticket)
INSERT INTO Ticket (Booking_ID, Ticket_Number, Issue_Date)
VALUES (5001, 'TKT-5001-DUP', '2026-10-23');

-- B10 1:1 rule. Expected: 1062 duplicate on Baggage.Booking_ID
INSERT INTO Baggage (Booking_ID, Baggage_Count, Total_Weight) VALUES (5001, 2,
20.00);
```

```sql

-- B11 Expected: 1062 duplicate Transaction_ID
INSERT INTO Payment (Booking_ID, Payment_Date, Payment_Method, Amount, Payment_Status, Transaction_ID)
VALUES (5002, '2026-10-23', 'Card', 5500.00, 'Successful', 'UPI-TXN-0001');

-- ---------- CHECK ----------
-- B12 Expected: 3819 chk_passenger_gender
INSERT INTO Passenger (First_Name, Last_Name, Email, Phone, Date_of_Birth, Gender)
VALUES ('Test', 'User', 'test3@example.com', '9000000004', '2000-01-01', 'X');

-- B13 Expected: 3819 chk_airport_terminals
INSERT INTO Airport (Airport_Code, Airport_Name, City, Country, Terminal_Count)
VALUES ('XYZ', 'Zero Terminal Airport', 'Pune', 'India', 0);

-- B14 Expected: 3819 chk_aircraft_seats
INSERT INTO Aircraft (Aircraft_Model, Registration_No, Total_Seats)
VALUES ('Ghost Jet', 'VT-NEG', -5);

-- B15 Expected: 3819 chk_aircraft_status
INSERT INTO Aircraft (Aircraft_Model, Registration_No, Total_Seats, Aircraft_Status)
VALUES ('Old Jet', 'VT-BRK', 100, 'Broken');

-- B16 Expected: 3819 chk_flight_airports (departure = arrival)
INSERT INTO Flight (Flight_Number, Aircraft_ID, Departure_Airport_ID, Arrival_Airport_ID,
                    Departure_DateTime, Arrival_DateTime)
VALUES ('AI111', 1, 1, 1, '2026-11-10 06:00:00', '2026-11-10 08:00:00');

-- B17 Expected: 3819 chk_flight_times (arrival before departure)
INSERT INTO Flight (Flight_Number, Aircraft_ID, Departure_Airport_ID, Arrival_Airport_ID,
                    Departure_DateTime, Arrival_DateTime)
VALUES ('AI112', 1, 1, 2, '2026-11-10 10:00:00', '2026-11-10 08:00:00');

-- B18 Expected: 3819 chk_flight_status
INSERT INTO Flight (Flight_Number, Aircraft_ID, Departure_Airport_ID, Arrival_Airport_ID,
Departure_DateTime, Arrival_DateTime, Flight_Status)
VALUES ('AI113', 1, 1, 2, '2026-11-11 06:00:00', '2026-11-11 08:00:00', 'Landed');

-- B19 Expected: 3819 chk_seat_class
INSERT INTO Seat (Aircraft_ID, Seat_Number, Seat_Class) VALUES (1, '15A', 'Premium');

-- B20 Expected: 3819 chk_booking_status
INSERT INTO Booking (Passenger_ID, Flight_ID, Seat_ID, Booking_Date, Booking_Status, Fare)
VALUES (1, 103, 3, '2026-10-23', 'Maybe', 4000.00);

```

```sql
-- B21 Expected: 3819 chk_booking_fare (negative fare)
INSERT INTO Booking (Passenger_ID, Flight_ID, Seat_ID, Booking_Date, Fare)
VALUES (1, 103, 3, '2026-10-23', -100.00);

-- B22 Expected: 3819 chk_ticket_status
INSERT INTO Ticket (Booking_ID, Ticket_Number, Issue_Date, Ticket_Status)
VALUES (5002, 'TKT-5002-AI502', '2026-10-23', 'Lost');

-- B23 Expected: 3819 chk_payment_method
INSERT INTO Payment (Booking_ID, Payment_Date, Payment_Method, Amount,
Payment_Status, Transaction_ID)
VALUES (5002, '2026-10-23', 'Cash', 5500.00, 'Successful', 'CASH-0001');

-- B24 Expected: 3819 chk_payment_amount (zero amount)
INSERT INTO Payment (Booking_ID, Payment_Date, Payment_Method, Amount,
Payment_Status, Transaction_ID)
VALUES (5002, '2026-10-23', 'UPI', 0, 'Pending', 'UPI-TXN-0002');

-- B25 Expected: 3819 chk_baggage_values (negative weight)
INSERT INTO Baggage (Booking_ID, Baggage_Count, Total_Weight) VALUES (5003, 1,
-4.00);

-- ---------- FOREIGN KEY: inserting a child with no parent ----------
-- B26 Expected: 1452 fk_flight_aircraft (aircraft 99 does not exist)
INSERT INTO Flight (Flight_Number, Aircraft_ID, Departure_Airport_ID,
Arrival_Airport_ID,
                    Departure_DateTime, Arrival_DateTime)
VALUES ('AI222', 99, 1, 2, '2026-11-12 06:00:00', '2026-11-12 08:00:00');

-- B27 Expected: 1452 fk_flight_arrival (airport 99 does not exist)
INSERT INTO Flight (Flight_Number, Aircraft_ID, Departure_Airport_ID,
Arrival_Airport_ID,
                    Departure_DateTime, Arrival_DateTime)
VALUES ('AI223', 1, 1, 99, '2026-11-12 06:00:00', '2026-11-12 08:00:00');

-- B28 Expected: 1452 fk_seat_aircraft
INSERT INTO Seat (Aircraft_ID, Seat_Number) VALUES (99, '1A');

-- B29 Expected: 1452 fk_booking_passenger
INSERT INTO Booking (Passenger_ID, Flight_ID, Seat_ID, Booking_Date, Fare)
VALUES (99, 103, 3, '2026-10-23', 4000.00);

-- B30 Expected: 1452 fk_booking_flight
INSERT INTO Booking (Passenger_ID, Flight_ID, Seat_ID, Booking_Date, Fare)
VALUES (1, 999, 3, '2026-10-23', 4000.00);

-- B31 Expected: 1452 fk_booking_seat
INSERT INTO Booking (Passenger_ID, Flight_ID, Seat_ID, Booking_Date, Fare)
VALUES (1, 103, 999, '2026-10-23', 4000.00);

-- B32 Expected: 1452 fk_ticket_booking
INSERT INTO Ticket (Booking_ID, Ticket_Number, Issue_Date) VALUES (9999, 'TKT-
9999', '2026-10-23');

```

```sql
-- B33 Expected: 1452 fk_payment_booking
INSERT INTO Payment (Booking_ID, Payment_Date, Payment_Method, Amount,
Payment_Status, Transaction_ID)
VALUES (9999, '2026-10-23', 'UPI', 100.00, 'Pending', 'UPI-TXN-9999');

-- B34 Expected: 1452 fk_baggage_booking
INSERT INTO Baggage (Booking_ID) VALUES (9999);

-- ---------- FOREIGN KEY: deleting / changing a parent that has children ----
------
-- B35 Expected: 1451 (passenger 1 still has bookings)
DELETE FROM Passenger WHERE Passenger_ID = 1;

-- B36 Expected: 1451 (booking 5001 still has a ticket, payment and baggage
record)
DELETE FROM Booking WHERE Booking_ID = 5001;

-- B37 Expected: 1451 (flight 101 still has bookings)
DELETE FROM Flight WHERE Flight_ID = 101;

-- B38 Expected: 1451 (seat 1 is used in bookings)
DELETE FROM Seat WHERE Seat_ID = 1;

-- B39 Expected: 1451 (airport 1 is used by flights; its key cannot be changed
either)
UPDATE Airport SET Airport_ID = 50 WHERE Airport_ID = 1;

-- ========================================================================
-- PART C : POSITIVE VERIFICATION (relationships and integrity hold)
-- ========================================================================

-- C1 Rows that survived: the failed statements in Part B must have changed
nothing
SELECT 'Passenger' AS Tbl, COUNT(*) AS Row_Count FROM Passenger UNION ALL
SELECT 'Airport', SELECT COUNT(*)     FROM     Airport UNION  ALL
'Aircraft', SELECT 'Flight', COUNT(*)    FROM     Aircraft UNION  ALL
SELECT   'Seat',   SELECT COUNT(*)      FROM      Flight UNION  ALL
'Booking',         SELECT COUNT(*)      FROM       Seat UNION  ALL
'Ticket',          SELECT COUNT(*)    FROM    Booking UNION  ALL
'Payment',         SELECT COUNT(*)      FROM      Ticket UNION  ALL
'Baggage',                COUNT(*)    FROM    Payment UNION ALL
                          COUNT(*) FROM Baggage;
-- Expected after Parts A and B: 2, 3, 3, 4, 5, 4, 1, 1, 2

-- C2 Full booking record joined across all nine tables (all relationships
work)
SELECT b.Booking_ID,
          CONCAT(p.First_Name,     '     ',     p.Last_Name)          AS Passenger,
          f.Flight_Number,
          dep.Airport_Code                                           AS From_Airport,
          arr.Airport_Code                                           AS To_Airport,
          f.Departure_DateTime,
          a.Aircraft_Model,
          s.Seat_Number, s.Seat_Class,
```

```
260             b.Fare, b.Booking_Status,
261             t.Ticket_Number, t.Ticket_Status,
262             pay.Payment_Method, pay.Amount, pay.Payment_Status,
263             bg.Baggage_Count, bg.Total_Weight
264 FROM Booking b
265 JOIN  Passenger  p  JOIN ON p.Passenger_ID = b.Passenger_ID
266 Flight f                  ON f.Flight_ID          = b.Flight_ID
267 JOIN  Airport  dep  JOIN ON dep.Airport_ID = f.Departure_Airport_ID
268 Airport  arr  JOIN  Aircraft ON arr.Airport_ID = f.Arrival_Airport_ID
269 a JOIN Seat s             ON a.Aircraft_ID = f.Aircraft_ID
270 LEFT JOIN Ticket t        ON s.Seat_ID            = b.Seat_ID
271                              ON t.Booking_ID        = b.Booking_ID
272 LEFT JOIN Payment pay ON pay.Booking_ID = b.Booking_ID
273 LEFT JOIN Baggage bg ON bg.Booking_ID = b.Booking_ID
274 WHERE b.Booking_ID = 5001;
275
276 -- C3 Orphan check: child rows with no parent. Every query must return0
    rows.
277 SELECT b.Booking_ID FROM Booking b LEFT JOIN Passenger p ON p.Passenger_ID                =
    b.Passenger_ID WHERE p.Passenger_ID IS NULL;
278 SELECT b.Booking_ID FROM Booking b LEFT JOIN Flight f                  ON f.Flight_ID       =
    b.Flight_ID          WHERE f.Flight_ID          IS NULL;
279 SELECT b.Booking_ID FROM Booking b LEFT JOIN Seat s                   ON s.Seat_ID         =
    b.Seat_ID            WHERE s.Seat_ID            IS NULL;
280 SELECT t.Ticket_ID FROM Ticket t LEFT JOIN Booking b                 ON b.Booking_ID      =
    t.Booking_ID         WHERE b.Booking_ID         IS NULL;
281 SELECT y.Payment_ID FROM Payment y LEFT JOIN Booking b                ON b.Booking_ID      =
    y.Booking_ID         WHERE b.Booking_ID         IS NULL;
282 SELECT g.Baggage_ID FROM Baggage g LEFT JOIN Booking b                ON b.Booking_ID      =
    g.Booking_ID         WHERE b.Booking_ID         IS NULL;
283
284 -- C4 Double-booking check: any seat sold twice on the same flight? Must
    return 0 rows.
285 SELECT Flight_ID, Seat_ID, COUNT(*) AS Times_Booked
286 FROM Booking
287 GROUP BY Flight_ID, Seat_ID
288 HAVING COUNT(*) > 1;
289
290 -- C5 Same seat on different flights is allowed (seat 1 appears on flights
    101 and 102)
291 SELECT Seat_ID, Flight_ID, Booking_ID FROM Booking WHERE Seat_ID = 1;
292
293 -- C6 Booking per passenger (one passenger : many bookings)
294 SELECT p.Passenger_ID, p.First_Name, COUNT(b.Booking_ID) AS Total_Bookings
295 FROM Passenger p LEFT JOIN Booking b ON b.Passenger_ID = p.Passenger_ID
296 GROUP BY p.Passenger_ID, p.First_Name;
297
298 -- C7 Inventory of every constraint stored in the database
299 SELECT TABLE_NAME, CONSTRAINT_NAME, CONSTRAINT_TYPE
300 FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS
301 WHERE TABLE_SCHEMA = 'airline_reservation_system'
302 ORDER BY TABLE_NAME, CONSTRAINT_TYPE, CONSTRAINT_NAME;
303
304 -- C8 Column defaults stored in the database
```

```sql
SELECT TABLE_NAME, COLUMN_NAME, COLUMN_DEFAULT
FROM    INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'airline_reservation_system' AND COLUMN_DEFAULT IS NOT NULL
ORDER BY TABLE_NAME, ORDINAL_POSITION;

```